

Proiect Procesarea Imaginilor
Sistem SLAM bazat pe ORB-SLAM3 și Vizualizare OpenCV

Student: Toth Antonio-Roberto

Mai, 2026

Cuprins

1	Introducere	2
1.1	Contextul temei	2
1.2	Problemele de rezolvat	2
1.3	Obiectivele propuse	2
1.4	Structura documentației	3
2	Fundamentare Teoretică	4
2.1	Documentare bibliografică	4
2.2	Metode utilizate în SLAM	4
2.3	Posibile soluții pentru implementare	5
3	Proiectare și Implementare	7
3.1	Detalierea Soluției Alese	7
3.2	Modulele de Implementare	7
3.2.1	Modulul SLAM Manager	7
3.2.2	Modulul MapVisualizer	7
3.3	Algoritmii Implementați	8
3.3.1	Algoritmul de Dilatare Manuală	8
3.3.2	Filtrarea Outlier-ilor și a Punctelor Indepărtate	8
3.4	Modul de Utilizare al Aplicației	8
3.5	Compilarea și Rularea Sistemului	9
3.5.1	Procesul de Build	9
3.5.2	Execuția Aplicației	9
3.6	Schema Bloc a Sistemului	9
3.7	Instalarea Proiectului	9
4	Testare și Validare	11
4.0.1	Detectarea caracteristicilor ORB	11
4.0.2	Vizualizarea traiectoriei	11
5	Concluzii	13
5.1	Sinteza Realizărilor	13
5.2	Eficiența Algoritmilor Implementați	13
5.3	Aplicații și Direcții Viitoare	13

Capitolul 1

Introducere

1.1 Contextul temei

În domeniul procesării imaginilor și al viziunii artificiale, estimarea poziției unei camere într-un mediu necunoscut reprezintă o problemă fundamentală, cu aplicații în robotică, realitate augmentată și sisteme autonome. O soluție pentru această problemă este reprezentată de tehnicile de tip SLAM (Simultaneous Localization and Mapping), care permit localizarea unui sistem în timp ce construiesc simultan o hartă a mediului.

În cadrul acestui proiect se utilizează biblioteca OpenCV pentru prelucrarea imaginilor și integrarea unui sistem SLAM bazat pe ORB-SLAM3

1.2 Problemele de rezolvat

Implementarea unui sistem SLAM implică mai multe provocări tehnice.

În primul rând, este necesară detectarea și urmărirea caracteristicilor relevante din imagini succesive.

În al doilea rând, trebuie estimată mișcarea camerei între cadre consecutive, utilizând metode robuste de calcul al transformărilor geometrice.

De asemenea, o problemă importantă este reprezentată de integrarea unui sistem SLAM existent într-o aplicație proprie, astfel încât să se poată extrage informațiile relevante (poziția camerei și punctele din hartă) și să se realizeze o vizualizare personalizată. Eliminarea dependențelor inutile (precum interfețele grafice externe) și adaptarea sistemului pentru utilizarea cu OpenCV constituie, de asemenea, un aspect esențial.

1.3 Obiectivele propuse

Obiectivul principal al proiectului este dezvoltarea unei aplicații capabile să estimeze traiectoria unei camere utilizând un sistem SLAM și să o vizualizeze într-o reprezentare bidimensională.

- Integrarea unui sistem SLAM funcțional (ORB-SLAM3) într-o aplicație proprie;
- Dezvoltarea unui modul intermediar (SLAM Manager) pentru gestionarea datelor;
- Extragerea poziției camerei în timp real;

- Obținerea și procesarea punctelor caracteristice din hartă;
- Realizarea unei vizualizări a traiectoriei și a punctelor detectate folosind OpenCV;

1.4 Structura documentației

Documentația este organizată în mai multe capitole. Capitolul următor prezintă fundamentele teoretice ale tehnicilor de tip SLAM și ale metodelor de procesare a imaginilor utilizate.

Ulterior, este descrisă arhitectura sistemului propus și modul de integrare a componentelor principale. În continuare, sunt prezentate detaliile de implementare, incluzând modul de utilizare al bibliotecilor și structura aplicației. Ultimele capitole sunt dedicate rezultatelor obținute, analizelor și concluziilor finale.

Capitolul 2

Fundamentare Teoretică

2.1 Documentare bibliografică

Problema localizării și cartografierii simultane (SLAM - Simultaneous Localization and Mapping) a fost intens studiată în ultimele decenii, fiind esențială în robotică și viziune artificială. Primele abordări au fost bazate pe metode probabilistice, precum filtrul Kalman extins (EKF-SLAM), care modelează incertitudinea poziției și a mediului [1].

Ulterior, au fost dezvoltate metode bazate pe particule, cum ar fi FastSLAM, care oferă o scalabilitate mai bună în medii complexe [2]. O evoluție majoră a fost apariția tehnicilor de tip visual SLAM, care utilizează exclusiv informația vizuală provenită de la camere [3].

Un rol esențial în aceste metode îl joacă detecția și descrierea caracteristicilor. Algoritmi consacrați precum SIFT [4] și SURF [5] au oferit robustețe la scalare și rotație, însă au un cost computațional ridicat. Pentru aplicații în timp real, a fost introdus algoritmul ORB (Oriented FAST and Rotated BRIEF), care oferă un compromis eficient între viteză și performanță [6].

Sistemele moderne de SLAM, precum ORB-SLAM [7], ORB-SLAM2 [8] și ORB-SLAM3 [9], utilizează caracteristici ORB pentru a realiza localizarea și maparea în timp real. ORB-SLAM3 reprezintă o extindere semnificativă, oferind suport pentru sisteme monoculare, stereo și RGB-D, precum și o acuratețe îmbunătățită în estimarea traiectoriei[9].

2.2 Metode utilizate în SLAM

Fluxul de procesare include:

1. **Detecția caracteristicilor:** Această etapă constă în identificarea punctelor distinctive din imagine (colțuri, muchii) și asocierea unor descriptori care permit recunoașterea lor în cadre succesive.

ORB este frecvent utilizat datorită:

- vitezei ridicate;
- robusteții la rotație;
- eficienței computaționale.

2. **Asocierea caracteristicilor (Feature Matching):** Caracteristicile detectate în cadre consecutive sunt comparate pentru a identifica corespondențe. Aceasta se face de obicei folosind distanțe între descriptori.
3. **Estimarea mișcării:** Pe baza corespondențelor dintre puncte, se calculează transformarea geometrică dintre cadre:
 - matrice esențială / fundamentală;
 - estimarea poziției și orientării.
4. **Reconstrucția hărții:** Pe baza punctelor observate din mai multe cadre, se reconstruiesc puncte ale mediului.
5. **Optimizarea hărții:** Pentru îmbunătățirea preciziei, se aplică metode de optimizare globală care ajustează simultan:
 - pozițiile camerelor;
 - pozițiile punctelor din hartă.

2.3 Posibile soluții pentru implementare

1. **Utilizarea OpenCV pentru un SLAM:** OpenCV oferă funcții pentru:
 - detectarea caracteristicilor (ORB, SIFT);
 - calculul transformărilor geometrice;
 - reconstrucție 3D de bază.

Avantaje:

- implementare mai simplă;
- flexibilitate.

Dezavantaje:

- nu oferă un sistem SLAM complet;
- necesită integrare manuală a componentelor.

2. **Integrarea unui SLAM existent (ORB-SLAM3):** Această abordare presupune utilizarea unui sistem SLAM complet și performant, precum ORB-SLAM3.

Avantaje:

- performanță ridicată;
- sistem deja optimizat;
- suport pentru mai multe tipuri de senzori.

Dezavantaje:

- complexitate la integrare;
- necesită adaptare pentru aplicația proprie

3. **Soluția aleasă:** În cadrul acestui proiect, s-a ales integrarea sistemului ORB-SLAM3 într-o aplicație proprie, utilizând OpenCV pentru vizualizare

Această soluție oferă un compromis optim între:

- performanță;
- complexitate la implementare;
- posibilitatea de personalizare.

Pentru gestionarea interacțiunii cu sistemul SLAM, se introduce un modul intermediar (SLAM Manager), responsabil pentru:

- preluarea poziției camerei;
- extragerea punctelor din hartă;
- furnizarea datelor către modulul de vizualizare.

Vizualizarea traiectoriei și a punctelor se realizează folosind funcțiile grafice din OpenCV. Fluxul datelor:

Frame - Slam - Poziția camerei - Manager - Vizualizare OpenCV

Capitolul 3

Proiectare și Implementare

3.1 Detalierea Soluției Alese

Soluția implementată se bazează pe integrarea nucleului de calcul al sistemului ORB-SLAM3 într-o arhitectură modulară C++, utilizând biblioteca OpenCV pentru procesarea finală și redarea grafică. Decizia de a dezactiva modulul de vizualizare nativ (Pangolin) a fost luată pentru a permite un control granular asupra modului în care datele spațiale sunt filtrate și proiectate într-o hartă 2D accesibilă.

3.2 Modulele de Implementare

Sistemul este structurat în jurul a trei componente software fundamentale, fiecare având un rol bine definit în fluxul de date:

3.2.1 Modulul SLAM Manager

Acest modul acționează ca o interfață între sistemul SLAM și restul aplicației. Rolul său principal este de a încapsula complexitatea bibliotecii ORB-SLAM3 și de a returna un obiect de tip `SLAMState` ușor de procesat.

- **Funcționalitate:** Primește cadre grayscale și timestamp-uri, invocând metoda `TrackMonocular`.
- **Gestionarea Poziției:** Calculează poziția camerei în coordonate globale prin inversarea matricei de transformare *World-to-Camera* (T_{cw}) obținută de la tracker-ul sistemului.
- **Extracția Datelor:** Interoghează Atlasul sistemului pentru a obține punctele de hartă active (`MapPoints`) și starea curentă a procesului de urmărire.

3.2.2 Modulul MapVisualizer

Reprezintă componenta de randare grafică custom. Acesta transformă coordonatele 3D furnizate de SLAM într-o reprezentare vizuală bidimensională (proiecție de tip *top-down*).

- **Harta de Densitate:** Utilizează o matrice de acumulare (`mAccumulated`) de tip `CV_32F` pentru a stoca frecvența punctelor detectate, permițând o reprezentare mai stabilă a mediului.

- **Sistemul de Coordonate:** Implementează o funcție de conversie `worldToCanvas` care mapează coordonatele reale (X, Z) pe pixelii ferestrei de afișare, utilizând un factor de scală și o origine centrată.

3.3 Algoritmii Implementați

În cadrul procesului de dezvoltare, au fost implementați și adaptați o serie de algoritmi pentru a asigura robustețea vizualizării:

3.3.1 Algoritmul de Dilatare Manuală

Pentru a compensa dimensiunea redusă a punctelor proiectate și a face harta lizibilă, a fost implementat un algoritm de dilatare bazat pe căutarea maximului local într-o vecinătate de tip nucleu (kernel).

- **Logică:** Pentru fiecare pixel din harta de densitate, algoritmul verifică o fereastră de dimensiune variabilă ($N \times N$) și atribuie pixelului central valoarea maximă găsită. Acest proces elimină discontinuitățile vizuale fine.

3.3.2 Filtrarea Outlier-ilor și a Punctelor Indepărtate

Un aspect critic solicitat a fost eliminarea punctelor de zgomot care nu au legătură cu traiectoria. Strategia de filtrare include:

1. **Criteriul Observațiilor:** Se utilizează metoda `mp->Observations()` pentru a păstra doar punctele care au fost validate în cel puțin 5 cadre diferite, eliminând astfel punctele "fantomă".
2. **Limitarea Spațială (Max Radius):** Se calculează distanța euclidiană față de camera curentă:

$$d = \sqrt{(x_{pt} - x_{cam})^2 + (y_{pt} - y_{cam})^2 + (z_{pt} - z_{cam})^2} \quad (3.1)$$

Punctele pentru care $d > 25.0$ sunt eliminate din fluxul de randare.

3.4 Modul de Utilizare al Aplicației

Aplicația este o utilitară în linie de comandă care necesită trei parametri de intrare:

1. **Vocabularul ORB:** Un fișier comprimat (`.tar.gz` sau `.txt`) care conține nodurile arborelui vizual pentru recunoașterea trăsăturilor.
2. **Fișierul de Configurare:** Conține parametrii intrinseci ai camerei și setările extractorului ORB.
3. **Folderul de Date:** Calea către secvența de imagini (format compatibil KITTI).

În timpul rulării, aplicația deschide două ferestre OpenCV:

- **"ORB-SLAM3 Features":** Afișează imaginea originală cu trăsăturile detectate marcate prin pătrate verzi.
- **"Map":** Afișează traiectoria în timp real (linie albastră) și punctele hărții (puncte roșii dilatate) pe un grid de referință.

3.5 Compilarea și Rularea Sistemului

Deoarece proiectul integrează biblioteci complexe precum *OpenCV*, *Eigen* și *Sophus*, procesul de build este gestionat prin *CMake*, asigurând portabilitatea și gestionarea corectă a dependențelor.

3.5.1 Procesul de Build

Pentru compilare, se utilizează următoarea secvență de comenzi în terminalul Linux:

```
mkdir build
cd build
./build.sh
```

3.5.2 Execuția Aplicației

După compilarea cu succes, executabilul rezultat poate fi lansat utilizând comanda de mai jos, respectând ordinea argumentelor definită în `main.cpp`:

```
./Proiect_PI path/to/ORBvoc.txt path/to/config.yaml path/to/
dataset/
```

Unde:

- `ORBvoc.txt`: Vocabularul vizual necesar pentru *re-localization* și *loop closure*.
- `config.yaml`: Parametrii de calibrare ai camerei (focal length, optical center) și pragurile extractorului ORB.
- `dataset/`: Folderul care conține imaginile și fișierul `times.txt` pentru sincronizare.

3.6 Schema Bloc a Sistemului

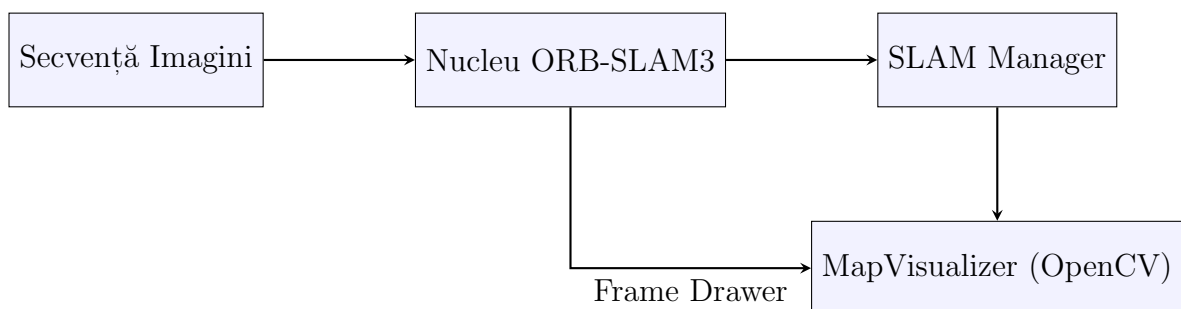


Figura 3.1: Schema fluxului de date și control al aplicației.

3.7 Instalarea Proiectului

Pentru instalarea și compilarea aplicației se execută următoarele comenzi în terminal:

```
# 1. Clonarea proiectului
git clone https://github.com/tothantonio/ORB_SLAM3.git

# 2. Accesarea folderului proiectului
cd ORB_SLAM3

# 3. Acordarea permisiunilor de executie
chmod +x build_prerequisites.sh
chmod +x build.sh

# 4. Instalarea dependențelor și compilarea proiectului
./build_prerequisites.sh
```

Scriptul `build_prerequisites.sh` realizează automat:

- instalarea dependențelor necesare;
- instalarea OpenCV 4.4;
- compilarea bibliotecilor Thirdparty;
- compilarea sistemului ORB-SLAM3;
- generarea executabilului final.

După compilarea cu succes a proiectului, aplicația poate fi rulată utilizând următoarea comandă:

```
./Examples/Proiect/slam_kitti \
Vocabulary/ORBvoc.txt \
Examples/Monocular/KITTI00-02.yaml \
dataset/
```

Unde:

- `ORBvoc.txt` reprezintă vocabularul ORB utilizat de sistem;
- `KITTI00-02.yaml` conține parametrii de configurare ai camerei;
- `dataset/` reprezintă folderul care conține imaginile utilizate pentru testare.

La rulare, aplicația deschide două ferestre OpenCV:

- **ORB-SLAM3 Features** – afișează caracteristicile detectate;
- **Map** – afișează traiectoria și punctele hărții.

Capitolul 4

Testare și Validare

Testele au fost efectuate pe secvențe monoculare din dataset-ul KITTI. Sistemul a reușit să urmărească poziția camerei în timp real și să genereze o hartă coerentă a mediului.

4.0.1 Detectarea caracteristicilor ORB

Figura următoare prezintă caracteristicile ORB detectate într-un cadru al secvenței video. Punctele verzi reprezintă trăsăturile utilizate pentru estimarea mișcării camerei.

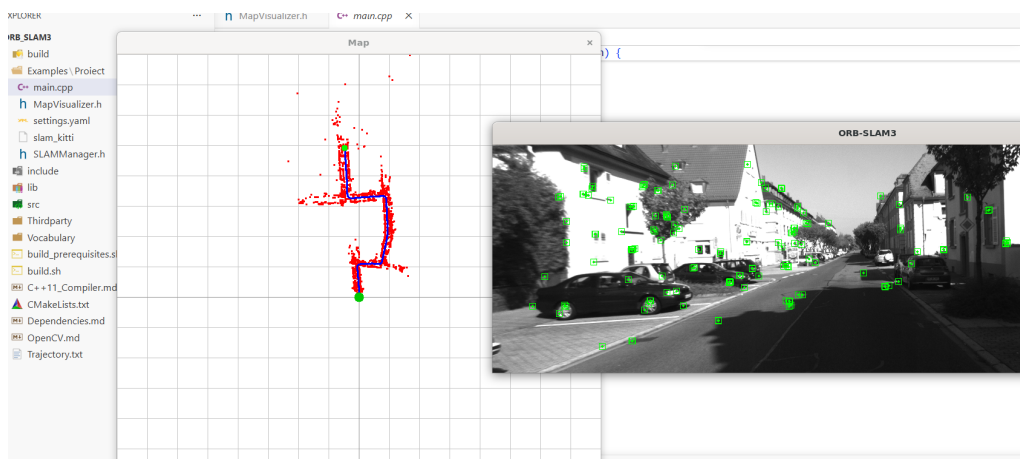


Figura 4.1: Caracteristici ORB detectate în imagine.

4.0.2 Vizualizarea traiectoriei

În timpul rulării, sistemul construiește o traiectorie estimată a camerei. Linia albastră reprezintă traseul parcurs, iar punctele roșii reprezintă punctele din hartă.

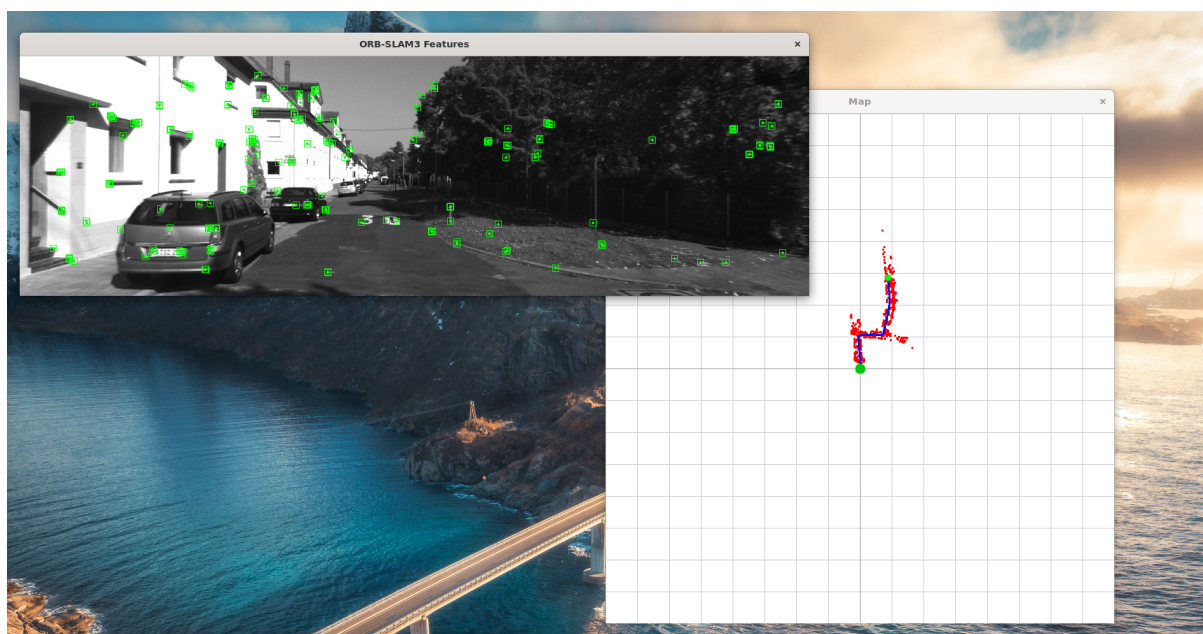


Figura 4.2: Traiectoria estimată și punctele hărții generate de sistem.

Capitolul 5

Concluzii

5.1 Sinteza Realizărilor

Proiectul a demonstrat succesul integrării nucleului computațional *ORB-SLAM3* într-o aplicație personalizată, demonstrând că este posibilă obținerea unui sistem de localizare și cartografiere performant prin utilizarea exclusivă a bibliotecii *OpenCV* pentru vizualizare. Prin eliminarea dependențelor grafice externe (Pangolin), s-a obținut o aplicație mai ușoară, cu un control total asupra fluxului de randare a datelor.

5.2 Eficiența Algoritmilor Implementați

Utilizarea tehnicilor de procesare a imaginilor a fost determinantă pentru calitatea reprezentării vizuale:

- **Filtrarea spațială:** Implementarea criteriului de distanță și a pragului minim de observații a eliminat eficient punctele de zgomot (*outliers*), rezultând o hartă curată, concentrată pe elementele geometrice relevante din proximitatea traiectoriei.
- **Algoritmul de dilatare:** Această tehnică a transformat punctele de hartă într-o reprezentare densă și intuitivă, facilitând interpretarea vizuală a structurii mediului parcurs prin intermediul funcțiilor OpenCV.

5.3 Aplicații și Direcții Viitoare

Arhitectura modulară bazată pe *SLAMManager* permite extinderea facilă a proiectului către sisteme multi-senzor. Ca direcții viitoare de dezvoltare, sistemul poate fi optimizat prin:

- **Integrarea senzorilor inerțiali (IMU):** Adăugarea datelor de la un accelerometru și giroscop ar permite un sistem *Visual-Inertial SLAM*. Aceasta ar rezolva limitările abordării monoculare, oferind o estimare corectă a scării metrice și menținerea urmăririi în momentele cu mișcări bruște sau iluminare slabă, unde algoritmul vizual ar putea eșua.
- **Integrarea GPS:** Utilizarea coordonatelor GPS ar permite ancorarea hărții locale într-un sistem de referință global. Fuziunea datelor SLAM cu GPS ar asigura o

precizie absolută pe distanțe lungi, eliminând complet deriva (*drift*) care rămâne inerentă oricărui sistem bazat exclusiv pe senzori locali.

În concluzie, lucrarea constituie o bază solidă pentru înțelegerea proceselor de viziune artificială de nivel înalt. Deși sistemul actual este pur vizual, performanțele obținute în estimarea traiectoriei și reconstrucția hărții validează arhitectura propusă ca un fundament viabil pentru sisteme de navigație complexe.

Bibliografie

- [1] H. Durrant-Whyte, T. Bailey, “Simultaneous Localization and Mapping: Part I”, IEEE Robotics Automation Magazine, 2006.
- [2] M. Montemerlo, S. Thrun, D. Koller, B. Wegbreit, “FastSLAM: A Factored Solution to the Simultaneous Localization and Mapping Problem”, AAAI, 2002.
- [3] A. J. Davison, “Real-Time Simultaneous Localization and Mapping with a Single Camera”, ICCV, 2003.
- [4] D. G. Lowe, “Distinctive Image Features from Scale-Invariant Keypoints”, IJCV, 2004.
- [5] H. Bay, T. Tuytelaars, L. Van Gool, “SURF: Speeded Up Robust Features”, ECCV, 2006
- [6] E. Rublee, V. Rabaud, K. Konolige, G. Bradski, “ORB: An Efficient Alternative to SIFT or SURF”, ICCV, 2011.
- [7] R. Mur-Artal, J. M. M. Montiel, J. D. Tardós, “ORB-SLAM: A Versatile and Accurate Monocular SLAM System”, IEEE Transactions on Robotics, 2015.
- [8] R. Mur-Artal, J. D. Tardós, “ORB-SLAM2: An Open-Source SLAM System for Monocular, Stereo and RGB-D Cameras”, IEEE Transactions on Robotics, 2017.
- [9] C. Campos, R. Elvira, J. J. Gómez Rodríguez, J. M. M. Montiel, J. D. Tardós, “ORB-SLAM3: An Accurate Open-Source Library for Visual, Visual-Inertial and Multi-Map SLAM”, IEEE Transactions on Robotics, 2021.